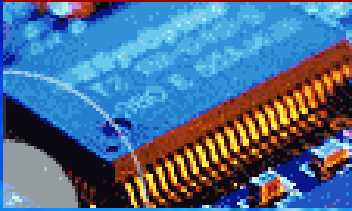
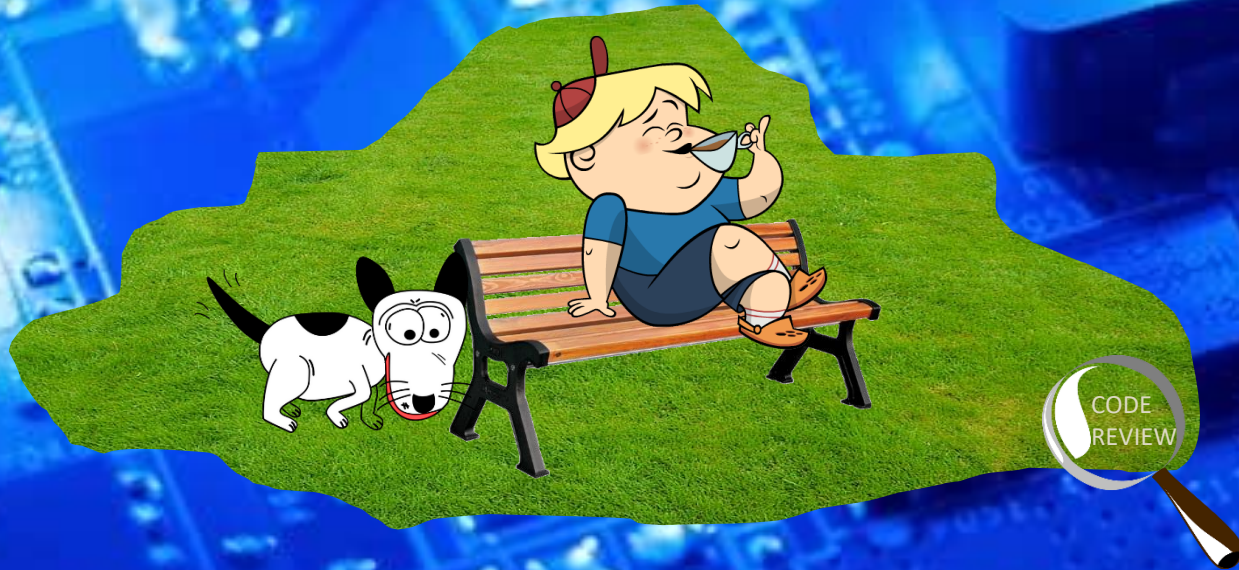


EEE3096S



**Embedded
Systems II**



Selection Process, Golden Measures, Intro to
Benchmarking & and Code Reviews

Lecture L04

Embedded Systems II

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Overview

- The selection process and need for benchmarking
- Golden Measures
- Benchmarking basics
- Benchmarking Pitfalls
- Code reviews as an approach to think about considering working in prac/project groups

Selection Process

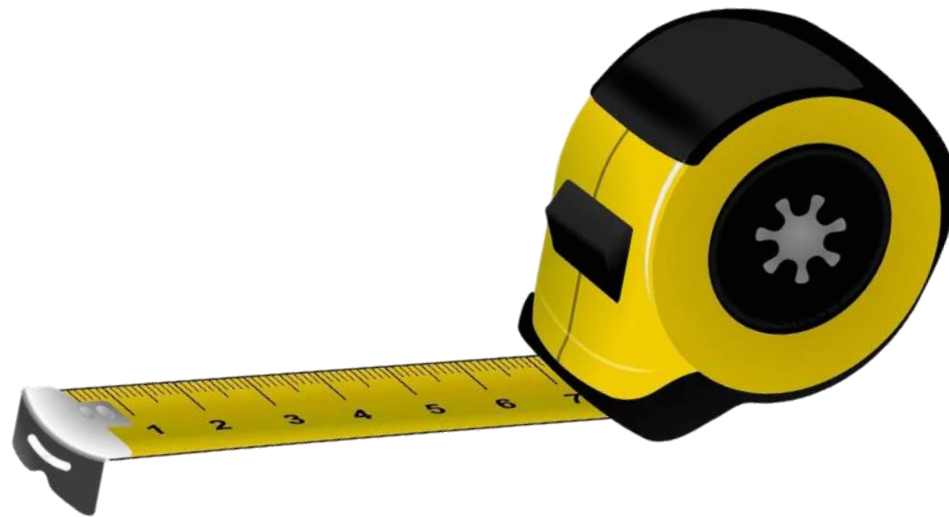
- **Selection Process =**
Selecting parts and other things from which you will build your embedded system
- This includes choosing processor(s), hardware components, software and code to incorporate and/or code or design aspects to reuse.



The Selection Process

- A significant part of the embedded design process
- Involves selecting the best software and hardware choices for the required **functionality**
- Correct selection is **critical to profit !\$!**
- Can (greatly) lower production costs
 - No money wasted on unused features
- Lower non-recurring engineering costs
 - Development cheaper with good tools
- Better reliability and maintainability

'Golden Measures'



Term: Golden Measure



- The Golden Measure concept:
 - A baseline benchmark solution (which is not about speed, but rather accuracy)
- The concept originates from theory of a 'gold standard measure'* which is:
 - A benchmark available under reasonable conditions. Not necessarily a perfect test, but the best available one that has a standard with known results. This is especially important when faced with the impossibility of direct measurements.

* This term originated more in medicine (although has a much older origin if looking further back) and is also widely applied to data science and computing nowadays.

Term: Golden Measure

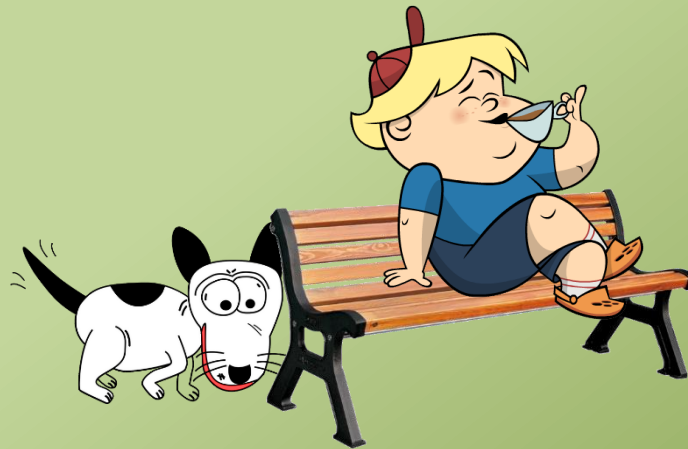


- Avoid confusion ...
 - Do not confuse with notion of a perfect physical measures, e.g. ‘perfect meter’, which is about getting a perfect instance of the thing you want to compare against (if such a thing exists it makes the process or comparison and measure easier and very reliable).
(<https://www.atlasobscura.com/places/the-last-original-standard-metre-paris-france>)
- A golden measure in computing can be:
 - A solution that may run slowly, isn’t optimized, but *you know* it gives (numerically speaking) excellent results

e.g., a solution written in MatLab, verify it is correct using graphs or exhaustive testing, or checking by hand with calculator, etc.

Benchmarking

Benchmarking Basics



Benchmarking



- **Benchmarking** =
Process of measuring performance of a system (an embedded computer system in our case)
- It's a program (or systematic approach) that **Quantitatively Evaluates** performance, cost and computer hardware and software resources (among other things) of a computing solution
- **Benchmark suite** =
A sets of benchmark programs designed to get a comprehensive view of the performance of a computer system for executing a variety of representative processing operations.

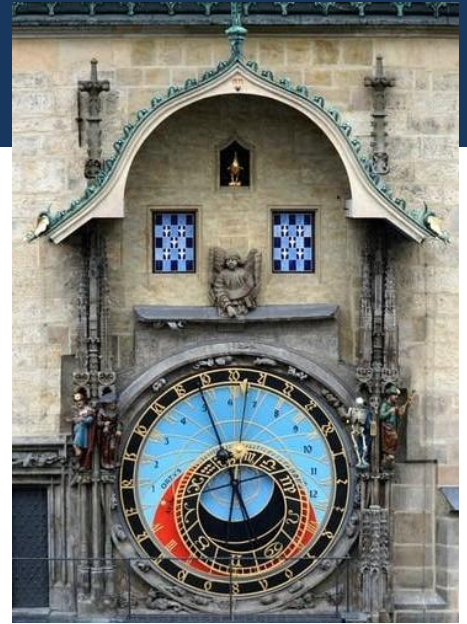
The Easiest, Usual Suspects



**For benchmarking software processor
performance**

These concepts link to the pracs for this course and also links later to the project

Wall Clock Time



Built-in timer with
added features

- Generally the most accurate*:
use a built-in timer, which is
directly related to **real time**
(e.g., if the timer measures 1s, then 1s
elapsed in the real world)

- Technique:

```
unsigned long long start; // store start time
unsigned long long end;   // store end time
start = read_the_timer(); // e.g. time()
    DO PROCESSNG
end = read_the_timer();   // e.g. time()
.. Output the time measurement (end-start), or save
it to an array if printing will interfere with the
times. Note: to avoid overflow, used unsigned vars.
```

See file:
Cycle.c



* from measuring processor speed for an easy human perspective on the issue at least

StdC: gettimeofday

gettimeofday

You might want to use this in pracs 😊

- Very portable, part of the StdC library
- Should be available on any Linux system
- Returns time in seconds and microseconds since midnight January 1 1970
- Uses struct timeval comprising
 - tv_sec : number of seconds
 - tv_usec : number of microseconds*
- Converting to microseconds will use huge numbers, rather work on differences

** Word of caution: some implementations always return 0 for the usec field!!*

On Cygwin, the resolution is only in milliseconds, so tv_usec in multiples of 1000. Not provided in DevC++ on Windows.

gettimeofday example

```
#include <stdio.h>
#include <sys/time.h>
#include <time.h>
struct timeval start_time, end_time; // variables to hold start and end time

int main() {
    int tot_usecs; /* may want to use long int on CPU with smaller word length */
    int i,j,sum=0;
    gettimeofday(&start_time, (struct timezone*)0); // starting timestamp

    /* do some work */
    for (i=0; i<10000; i++)
        for (j=0; j<i; j++) sum += i*j;

    gettimeofday(&end_time, (struct timezone*)0); // ending timestamp

    tot_usecs = (end_time.tv_sec-start_time.tv_sec) * 1000000 +
                (end_time.tv_usec-start_time.tv_usec);
    printf("Total time: %d usec.\n", tot_usecs);
}
```



See **timing.c** code file

Clock Wrapping

- While this issue isn't really a problem with `gettimeofday`, it is a problem when using a high-resolution time of limited length that might wrap (i.e. go from high value to low value) in the code section timed. Assuming that an unsigned integer is being used.
- The solution is doing the time duration calculation using AND or modula, e.g.:

`elapsed = (end - start) & MAXINT`

can use `UINT32_MAX = 0xFFFFFFFF` for 32-bit uint (or `uint32_t`)



Examples

Quick look at some code examples ...

Available on Amathuba .. EEE3096S Module 2:

 EEE3096S-L04
Benchmarking

 Timer (tic toc in
C)

 solution_using
clock_ge_time

 timer_wrapping

Benchmarking Pitfalls



What is wrong about using only wall clock time?



It can provide a false impression of how effective your solution is – or rather more correctly it at least **doesn't give a 'full picture' of performance** ...

Recommended tips for using wall-clock time:

- Typically do tests after the system has 'warmed up' (cache loaded) by running the same data multiple times*
- May show the solution is quicker... but at what costs? e.g.:
 - Speed improved but accuracy sacrificed?
 - Development effort vs. execution speed improvement?
 - Resource costs for upgrading vs. costs saved by remaining with the old version?
 - Power usage? Does the new solution need more power (per execution, also on average including idle time)
 - Maintainability? (e.g. is the new version more complex?)
 - Environment impact? (Does the upgrade result in waste that could be environmentally detrimental)

* But note, this could also be a false impression depending how your system is likely to work, if operations are typically not going to be in cache when invoked, this approach could invalidate your benchmarking.

Benchmarking: what to test

- What can be benchmarked? For DSPs or embedded micros...
- Compiler
 - Converts High Level Language to Assembly language thus we benchmark compiler efficiency (such as how efficient is the generated assembly code?)
- The Processor
 - Code in hand-crafted/inspected assembly (to make comparisons fair)
- Operating System
 - Interrupt latencies, overhead of operating system calls, limits on devices, kernel size, availability of services and facilities such as support for virtual memory and paged memory.
- Platform
 - Scalability of memory. Peripheral limits. Interfaces supported. Power use. Power saving features. OS's supported.
- Applications

Benchmarking: what is usually measured

- For embedded and DSP systems we typically benchmark:
 - Cycle count
 - Data and Program memory usage
 - Execution time
 - Power consumption (has recently become a common thing to report on)

Issue #1: Processor Performance

- This is usually talked about the most, and is often the most important design decision
- *Particularly:* is the processor/controller/DSP we're planning be fast enough?
- Performance measuring tools are available to analyze this, at least roughly, particularly:
 - MIPS (Million Instructions Per Second)
 - FLOPS (Floating Point MIPS)
 - Dhrystone Benchmark

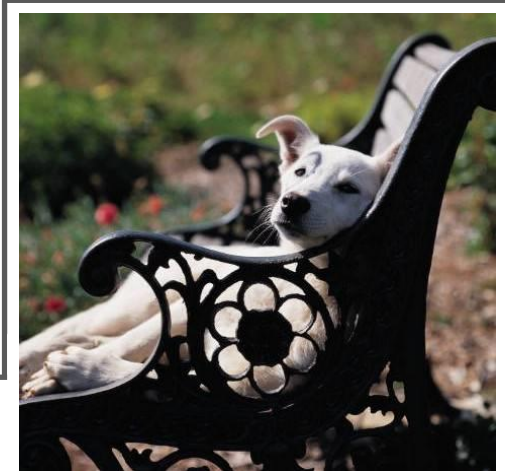
An important point to note: that processor manufacturer might optimize their toolchains to get better benchmark results using these common tools, so it's usually best to use some more representative tests for your particular application.



Examples of Standard Benchmarks

- The Dhrystone benchmark
 - Developed in 1984 by R. Wecker
 - Tests integer performance only
 - No I/O, no OS calls, no floating point
- Good for estimating integer performance
- Dhrystone score is the number of times a small program can run per second
- Like all benchmarks, applicability of dhrystone varies (may be totally irrelevant to your specific application)

... so don't get too comfortable using a common very 'general purpose' benchmarking tool like Dhrystone!



Meaningful Benchmarking

- How will the processor perform with our application?
- Including complex factors
 - Interrupt handling
 - Task switching
 - Memory performance
- Even more important for Real Time applications
 - Context switching time is critical
 - Save registers, change mode, etc.

EEMBC

Not Examined

- EDN* Embedded Micro Benchmark Consortium
- 46 test split into 5 suites
 - Automotive, consumer, networking, office automation, telecommunications
- Advantages
 - Speed and size statistics
 - Tailored to real application domains
 - Well designed, tested and documented
- Disadvantages
 - Expensive

* EDN: *Electronics Design News* publication and information company

See: www.edn.com

Running Custom Benchmarks

- Typically use evaluation boards
 - Save costs of designing support hardware
- Evaluation boards
 - Single board computers
 - Include a selection of common peripherals
- Some manufacturers sell **Hot Boards**
 - Specifically designed to boost benchmarks
 - Watch out for these boards



Bad News About Benchmarks

- Benchmarks do not consider all design factors which have an effect on performance, such as:
 - Power consumption
 - Physical size
 - Board layout
- Do not rely entirely on benchmark results for processor selection

Your benchmark results may look high and mighty – and sit above the rest of flock – but in reality you might just have seen part of the story; it may spend most time lower down 😊



Thinking Activity / Test Prep.

Consider that you are wanting to use an embedded operating system for a new product.



New product

Q: What are some of the factors that would influence your choice of operating system?

Consider discussing this in small groups or with a study buddy.

Strategy Tips: Need to think of the functionality and components involved, and from there decide OS needs, performance needs, etc.

I'll share some insight on this next week when you've hopefully thought over it on your own.

Well. I think I want an OS that is reliable...



An ingredient of a professional engineer

EEE3095S (CS) Students
Take Note re Project!

The Code Review



*These are things you should be aware of, and helps with thinking
and being prepared for pracs or projects
(in 2024 no EEE3096S project, but is **project** for EEE3095S!)*

What is a Code Review?

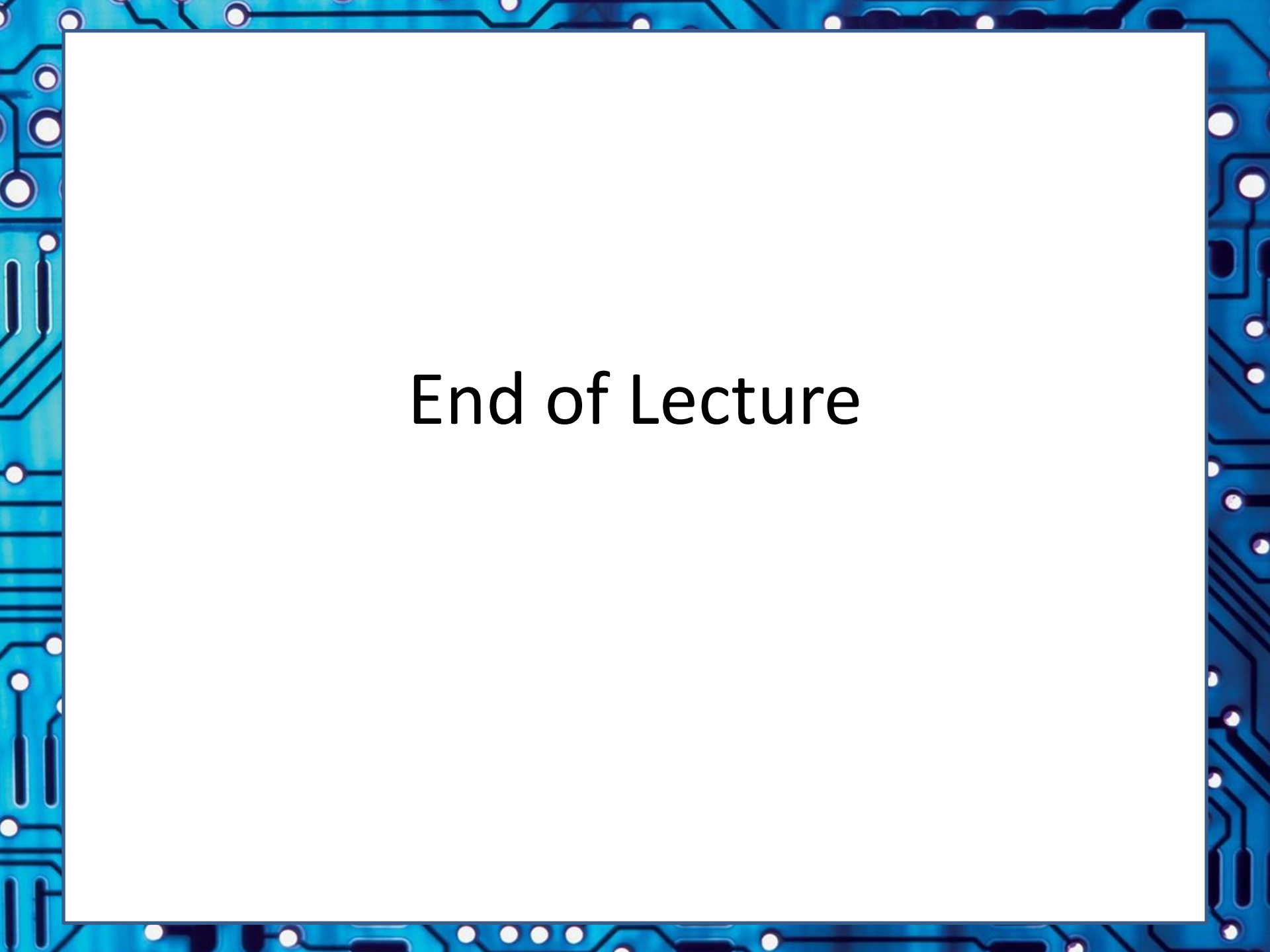
- It is a practice (e.g. meeting or regular team-link up) of reviewing code developed for a project
- It follows (generally) a systematic practice
(formulate your own initiative method for now!)
- It's about managing quality of code and ways to assess this, e.g. consider:
 - How to spot and log any defects
 - How many or common are the defects
 - Methods to use, such as (not necessarily all of these, not necessarily used in every review) are:
 - Unit testing
 - Function testing
 - Integration testing
 - How can this be improved?

Why Mention Code Reviews?



A best practice, commonly used in industry

Might also improve the mark you get for assignments



End of Lecture